Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

1η Εργαστηριακή Άσκηση

ΒΕΛΤΙΣΤΟΠΟΙΗΣΗ ΑΛΓΟΡΙΘΜΩΝ ΓΙΑ ΧΑΜΗΛΗ ΚΑΤΑΝΑΛΩΣΗ ΕΝΕΡΓΕΙΑΣ ΚΑΙ ΥΨΗΛΗ ΑΠΟΔΟΣΗ

Μάθημα: Σχεδιασμός Ενσωματωμένων Συστημάτων

Εξάμηνο: 9ο

Ονοματεπώνυμα: Μιχαλακόπουλος Δημήτριος (ΑΜ: 03120229), Γκενάκου Ζωή (ΑΜ: 03120015)

1.2 Ζητούμενο 1ο: Loop Optimization και Design Space Exploration

Το ζητούμενο σε αυτό το project αφορά την ανάλυση και την βελτιστοποίηση του αλγορίθμου `phods.c` στον μικροελεγκτή Renesas S7G2. Η βελτιστοποίηση του αλγορίθμου επιτυγχάνεται με στοχευμένες αλλαγές κώδικα μέσω της τεχνικής **Loop Optimization** και ο χρόνος εκτέλεσής του υπολογίζεται χρησιμοποιώντας μέτρηση κύκλων του επεξεργαστή (CPU). Στην συνέχεια, εφαρμόζουμε **Design Space Exploration** για διαφορετικές περιπτώσεις του μεγέθους μπλοκ προκειμένου να βρούμε τις βέλτιστες τιμές και να εξασφαλίσουμε κέρδος επιδόσεων.

1. Ανατρέχουμε στο datasheet του Renesas S7G2 και λαμβάνουμε τα εξής χαρακτηριστικά:

   - 4 MB Code Flash
   - 64 KB Data Flash
   - 640 KB SRAM
   - 8 KB Standby SRAM
   - Flash Cache (FCACHE)
   - Option-Setting Memory

Η λειτουργία καθενός από τα παραπάνω χαρακτηριστικά εξηγείται παρακάτω:

Code Flash: μη-πτητική μνήμη για μακροχρόνια αποθήκευση κώδικα. Εκεί αποθηκεύεται και εκτελείται ο κώδικας προγράμματος (firmware) του μικροελεγκτή.

Data Flash: μη-πτητική μνήμη για αποθήκευση δεδομένων ή/και ρυθμίσεων που διατηρούνται ακόμα και μετά την απενεργοποίηση του μικροελεγκτή.

SRAM: κύρια μνήμη με γρήγορη πρόσβαση για προσωρινή αποθήκευση και επεξεργασία δεδομένων κατά την εκτέλεση του προγράμματος.

Standby RAM: μνήμη που διατηρείται ενεργή σε λειτουργία χαμηλής κατανάλωσης ισχύος για την διατήρηση δεδομένων σε κατάσταση αδράνειας.

Flash Cache: μνήμη που διαχειρίζεται από το υλικό και αποθηκεύει συχνά χρησιμοποιούμενα δεδομένα από την μνήμη flash, μειώνοντας έτσι τον χρόνο πρόσβασης σε αυτή.

Option-Setting Memory: μνήμη που αποθηκεύει ρυθμίσεις συστήματος και παραμέτρους ασφαλείας, καθορίζοντας τον τρόπο εκκίνησης και ασφάλειας του μικροελεγκτή.

Τέλος, μέσω του e2studio τοποθετήσαμε ένα watch expression στο `SystemCoreClock` και μετά από εκτέλεση του κώδικά διαπιστώσαμε ότι η συχνότητα του ρολογιού που θα προγραμματιστεί του board είναι τα **240 MHz**.

2. Προτού ασχοληθούμε με τα ζητήματα της άσκησης πρέπει να κάνουμε δύο αλλαγές ώστε να πραγματοποιείται σωστά η εκτέλεση του κώδικα.

Στον υπολογισμό της μεταβλητής `disty` είναι λάθος η μεταβλητή **p2** και στην θέση της πρέπει να χρησιμοποιηθεί η μεταβλητή **q2** άρα ο κώδικας μετατρέπεται ως εξής:

```
        /*For all candidate blocks in Y dimension*/
        for(i=-S; i<S+1; i+=S)
        {
          disty = 0;

          /*For all pixels in the block*/
          for(k=0; k<B; k++)
          {
            for(l=0; l<B; l++)
            {
              p1 = current[B*x+k][B*y+l];

              if((B*x + vectors_x[x][y] + k) <0 ||
                  (B*x + vectors_x[x][y] + k) > (N-1) ||
                  (B*y + vectors_y[x][y] + i + l) < 0 ||
                  (B*y + vectors_y[x][y] + i + l) > (M-1))
              {
                q2 = 0;
              } else {
                q2 = previous[B*x+vectors_x[x][y]+k][B*y+vectors_y[x][y]+i+l];
              }

              if(p1-q2>0){
                   disty += p1-q2;
               }
               else{
                   disty += q2-p1;
               }
            }
          }
```

Προσπαθώντας να τρέξουμε την συνάρτηση `phods_motion_estimation` εμφανίζεται compilation error καθώς στα ορίσματά της οι πίνακες **current** και **previous** είναι δύο διαστάσεων, ενώ στο αρχείο `images.h` οι ίδιοι πίνακες είναι σε μονοδιάστατη μορφή. Επομένως, υπάρχει η ανάγκη δημιουργίας μια συνάρτησης που θα μετατρέπει τους μονοδιάστατους πίνακες σε δισδιάστατους. Την συνάρτηση αυτή την ονομάσαμε `convert1Dto2D` και η υλοποίησή της φαίνεται παρακάτω:

```
void convert1DTo2D(const int array[], int array_correct[N][M]) {
   int index_1d = 0;
   for (int i = 0; i < N; ++i) {
       for (int j = 0; j < M; ++j) {
           array_correct[i][j] = array[index_1d++];
       }
   }
}
```

Μετά από αυτές τις αλλαγές είμαστε έτοιμοι να λύσουμε τα ζητήματα της άσκησης. Προκειμένου να δούμε πόσο χρόνο θέλει κάθε εκτέλεση της συνάρτησης `phods_motion_estimation` θα μετράμε του κύκλους του CPU πριν και μετά την εκτέλεση της συνάρτησης θέτοντας την τιμή του register **DWT->CYCCNT** σε δύο μεταβλητές `start_cyc` και `end_cyc` αντίστοιχα. Αφού γίνουν 10 εκτελέσεις, διαιρούμε τους κύκλους με την συχνότητα υπολογίζοντας τον μέσο όρο (`average_time`), τον ελάχιστο (`min_time`) και τον μέγιστο (`max_time`) χρόνο εκτέλεσης του κώδικα χωρίς κανένα μετασχηματισμό. Ο κώδικας βρίσκεται στο αρχείο `base_phods.c` και φαίνεται παρακάτω:

```
void hal_entry(void) {
  // Code to initialize the DWT->CYCCNT register
  CoreDebug->DEMCR |= 0x01000000;
  ITM->LAR = 0xC5ACCE55;
  DWT->CYCCNT = 0;
  DWT->CTRL |= 1;
  /* Initialize your variables here */
  uint32_t start_cyc, end_cyc, run_cyc, total_cyc=0, min=UINT32_MAX, max=0;
  int previous1[N][M], current1[N][M],motion_vectors_x[N/B][M/B],
motion_vectors_y[N/B][M/B], i, j;
  convert1DTo2D(previous2, previous1);
  convert1DTo2D(current2, current1);
  for (int run=0; run <10; run ++) {
    /* Add timer code here */
    start_cyc = DWT->CYCCNT;
    phods_motion_estimation(current1, previous1, motion_vectors_x,
                            motion_vectors_y);
    end_cyc = DWT->CYCCNT;
    run_cyc = end_cyc - start_cyc;
    total_cyc += run_cyc;
    if (run_cyc > max){
        max=run_cyc;
    }
    if (run_cyc<min){
        min=run_cyc;
    }
  }
```

```
  float average_time= (float)total_cyc/10/CPU_FREQ;
  float min_time = (float)min / CPU_FREQ;
  float max_time = (float)max / CPU_FREQ;
}
```

Οι χρόνοι εκτέλεσης που προέκυψαν από το πρόγραμμα φαίνονται στον παρακάτω πίνακα:

ΤΙΜΗ	ΧΡΟΝΟΣ
Minimum	1.82112085 ms
Average	1.8211212 ms
Maximum	1.82112504 ms

3. Σε αυτό το ερώτημα θέλουμε να βελτιστοποιήσουμε τον αρχικό κώδικα ως προς τον χρόνο εκτέλεσης. Θα χρησιμοποιήσουμε **loop transformations** με σκοπό την αξιοποίηση του spatial locality, θα εφαρμόσουμε την αρχή της επαναχρησιμοποίησης δεδομένων (data reuse) και θα προσέξουμε ώστε οι χρονοβόροι υπολογισμοί να εκτελούνται μόνο μια φορά όπου είναι δυνατόν. Ο βέλτιστος κώδικας στο σύνολό του βρίσκεται στο αρχείο `best_phods.c`.

Αρχικά, παρατηρούμε ότι οι πράξεις που περιλαμβάνουν μεταβλητές του εξωτερικού loop (i, j) μπορούν να ενσωματωθούν στο επόμενο loop (x, y), επομένως, για να μειωθεί η πρόσθετη επιβάρυνση και να κανονικοποιηθεί η δομή του αλγορίθμου, εφαρμόζουμε loop merging ως εξής:

```
/*Initialize the vector motion matrices*/
for (i = 0; i < N / B; i++) {
  for (j = 0; j < M / B; j++) {
    vectors_x[i][j] = 0;
    vectors_y[i][j] = 0;
  }
}

/*For all blocks in the current frame*/
for (x = 0; x < N / B; x++) {
  for (y = 0; y < M / B; y++) {
    S = 4;

    while (S > 0) {···
    }
  }
}
}
```



```
/*For all blocks in the current frame*/
for (x = 0; x < N; x++) {
  for (y = 0; y < M; y++) {
      vectors_x[x][y] = 0;
      vectors_y[x][y] = 0;
      S = 4;

    while (S > 0) {···
    }
  }
}
}
```

Έπειτα, παρατηρούμε ότι μεταξύ των loops των αξόνων X και Y δεν παρατηρούμε κάποια εξάρτηση, επομένως εφαρμόζουμε πάλι loop merging ως εξής:

```
/*For all candidate blocks in X dimension*/
for (i = -S; i < S + 1; i += S) {
  distx = 0;

  /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }

  if (distx < min1) {···
  }
}

/*For all candidate blocks in Y dimension*/
for (i = -S; i < S + 1; i += S) {
  disty = 0;

  /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }

  if (disty < min2) {···
  }
}
```



```
for (i = -S; i < S + 1; i += S) {
  distx = 0;
  disty = 0;

  /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }

  if (distx < min1) {···
  }
                /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }

  if (disty < min2) {···
  }
}
```

Τέλος, εφαρμόζουμε ακόμα ένα loop merge, καθώς τα loops k και l δεν έχουν κάποια εξάρτηση, άρα η υλοποίησή μας θα πάρει την εξής μορφή:

```
for (i = -S; i < S + 1; i += S) {
  distx = 0;
  disty = 0;

  /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }

  if (distx < min1) {···
  }
            /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }

  if (disty < min2) {···
  }
}
```



```
/*For all candidate blocks in X dimension*/
for (i = -S; i < S + 1; i += S) {
  distx = 0;
  disty = 0;

  /*For all pixels in the block*/
  for (k = 0; k < B; k++) {···
  }
  if (distx < min1) {···
  }
  if (disty < min2) {
  }
```

Επίσης, παρατηρούμε ότι η εντολή p1 = current[B*x + k][B*y + l]; εκτελείται δύο φορές, οπότε μπορούμε να σβήσουμε το δεύτερο instance και ότι μπορεί να γίνει η παρακάτω βελτιστοποίηση:

```
if (p1 - p2 > 0) {
  distx += p1 - p2;
} else {
  distx += p2 - p1;
}
```

```
if (p1 - q2 > 0) {
  disty += p1 - q2;
} else {
  disty += q2 - p1;
}
```



```
distx += abs(p1 - p2);
disty += abs(p1 - q2);
```

Στην συνέχεια, θα προσπαθήσουμε να μειώσουμε τους χρονοβόρους υπολογισμούς δεδομένων που πραγματοποιούνται ξανά και ξανά. Έτσι, θέτουμε:

```
int vector_Bx = vectors_x[x][y] + B*x;
int vector_By = vectors_y[x][y] + B*y;
```

με αποτέλεσμα ο κώδικας να πάρει την εξής μορφή:

```
/*For all blocks in the current frame*/
for (x = 0; x < NdivB; x++) {
  for (y = 0; y < MdivB; y++) {
      vectors_x[x][y] = 0;
      vectors_y[x][y] = 0;
      S = 4;

    while (S > 0) {
      int vector_Bx = vectors_x[x][y] + B*x;
      int vector_By = vectors_y[x][y] + B*y;

      min1 = mul;
      min2 = mul;

      /*For all candidate blocks in X dimension*/
      for (i = -S; i < S + 1; i += S) {···
      }

      S = S / 2;
      vectors_x[x][y] += bestx;
      vectors_y[x][y] += besty;
    }
  }
}
```

Στο loop i για να μειώσουμε τους επαναλαμβανόμενους υπολογισμούς μπορούμε να χρησιμοποιήσουμε τις παρακάτω μεταβλητές:

```
int vector_Bxi = vector_Bx + i;
int vector_Byi = vector_By + i;
```

Για ακριβώς τον ίδιο λόγο στο loop k και στο loop l χρησιμοποιήσαμε τις παρακάτω μεταβλητές:

```
int vector_Bxik = vector_Bxi + k;
int vector_Bxk = vector_Bx + k;
int vector_Byil = vector_Byi + l;
int vector_Byl = vector_By + l;
```

Τέλος, έχοντας ως στόχο το data reuse κάτω από τα definitions των N και M υπολογίσαμε τις παρακάτω μεταβλητές: `const int NdivB=N/B, MdivB=M/B, Nminus1=N-1, Mminus1=M-1, mul=255*B*B;`

Οι χρόνοι εκτέλεσης που προέκυψαν από το πρόγραμμα φαίνονται στον παρακάτω πίνακα:

ΤΙΜΗ	ΧΡΟΝΟΣ
Minimum	0.737999973 ms
Average	0.738000381 ms
Maximum	0.738004164 ms

Συγκρίνοντας αυτά τα αποτελέσματα με τα αποτελέσματα του ερωτήματος 2 παρατηρούμε ότι έχουμε σημαντικές ελαττώσεις στους χρόνους εκτέλεσης του προγράμματος όπως φαίνεται παρακάτω:

$$\Delta t_{min} = t_{min,best_phods} - t_{min,base_phods} = 0.737999973\ ms - 1.82112085\ ms = -1.083120877\ ms$$
$$\Delta t_{av} = t_{av,best_phods} - t_{av,base_{phods}} = 0.738000381\ ms - 1.8211212\ ms = -1.083120819\ ms$$
$$\Delta t_{max} = t_{max,best_phods} - t_{max,base_phods} = 0.738004164\ ms - 1.82112504\ ms = -1.083120876\ ms$$

Η υπόδειξη στην εκφώνηση της άσκησης ότι ορισμένες βελτιστοποιήσεις δεν βελτιώνουν το performance σχετίζεται με τη χρήση της μνήμης Flash, όπου μόνο οι const μεταβλητές αποθηκεύονται. Αυτό σημαίνει ότι οι μη-const μεταβλητές γράφονται στη RAM, η οποία είναι πιο αργή από τη Flash και απαιτεί περισσότερη ενέργεια για προσπέλαση και εγγραφή.

Αυτό συμβαίνει γιατί η πρόσβαση στη Flash είναι συνήθως γρηγορότερη και ενεργειακά αποδοτικότερη για ανάγνωση στα ενσωματωμένα συστήματα, ενώ η RAM καταναλώνει περισσότερη ενέργεια, ιδιαίτερα για μεταβλητές που αλλάζουν συχνά. Έτσι, ορισμένες βελτιστοποιήσεις ενδέχεται να μην είναι αποτελεσματικές αν δεν μετακινούν δεδομένα σε const μεταβλητές, οι οποίες επωφελούνται από την ταχύτερη και πιο αποδοτική μνήμη Flash. Για αυτό τον λόγο ορισμένες μεταβλητές οι οποίες δεν αλλάζει η τιμή τους, επιλέγουμε να τις ορίσουμε ως const.

4. Στον κώδικα που προέκυψε από το προηγούμενο ερώτημα θέλουμε να κάνουμε Design Space Exploration προκειμένου να βρούμε για ποια τιμή B πετυχαίνουμε βέλτιστο block size. Έτσι, θα πραγματοποιήσουμε εξαντλητική αναζήτηση για όλες τις τιμές B που είναι κοινοί διαιρέτες του N και του M. Αφού έχουμε N=10 και M=10, τότε οι δυνατές τιμές του B είναι: 1, 2, 5, 10. Τρέχουμε λοιπόν το πρόγραμμα `sqr_phods.c` για αυτές τις τιμές και έτσι προκύπτει ο παρακάτω πίνακας:

ΧΡΟΝΟΣ	Minimum	Average	Maximum
B=1	1.23679172 ms	1.23679242 ms	1.23679999 ms
B=2	0.848137483 ms	0.848138356 ms	0.848145806 ms
B=5	0.738008355 ms	0.738009578 ms	0.738020812 ms
B=10	0.681566657 ms	0.681567879 ms	0.681579171 ms

Παρατηρούμε ότι όσο μεγαλώνουμε το block size, ο χρόνος εκτέλεσης του προγράμματος ελαττώνεται. Από την μία αυτό οφείλεται στο γεγονός ότι ο κώδικας αξιοποιεί καλά την **χωρική τοπικότα** (spatial locality) κατά την επεξεργασία των δεδομένων που βρίσκονται εντός του ίδιου μπλοκ. Από την άλλη, όμως, όταν προχωράμε στο επόμενο μπλοκ δεν έχουμε καλή **επαναχρησιμοποίηση δεδομένων** (data reuse). Από τα παραπάνω καταλήγουμε στο συμπέρασμα ότι όσο αυξάνουμε το block size του αλγορίθμου, αυξάνεται ο αριθμός των hits στην cache και έτσι μειώνονται τα conflict misses.

5. Στον κώδικα του ερωτήματος 3 θα εφαρμόσουμε πάλι Design Space Exploration προκειμένου να βρούμε για ποιες τιμές Bx και By πετυχαίνουμε βέλτιστο block size. Αυτή την φορά η εξαντλητική αναζήτηση θα πραγματοποιηθεί για όλα τα Bx που είναι διαιρέτες του N και για όλα τα By που είναι διαιρέτες του M, άρα για τιμές Bx: 1, 2, 5, 10 και By: 1, 2, 5, 10. Τρέχουμε λοιπόν το πρόγραμμα `rect_phods.c` για αυτές τις τιμές και έτσι προκύπτει ο παρακάτω πίνακας:

ΧΡΟΝΟΣ		Minimum	Average	Maximum
Bx=1	By=1	1.23679172 ms	1.23679242 ms	1.23679999 ms
Bx=1	By=2	0.931037473 ms	0.931038754 ms	0.931049988 ms

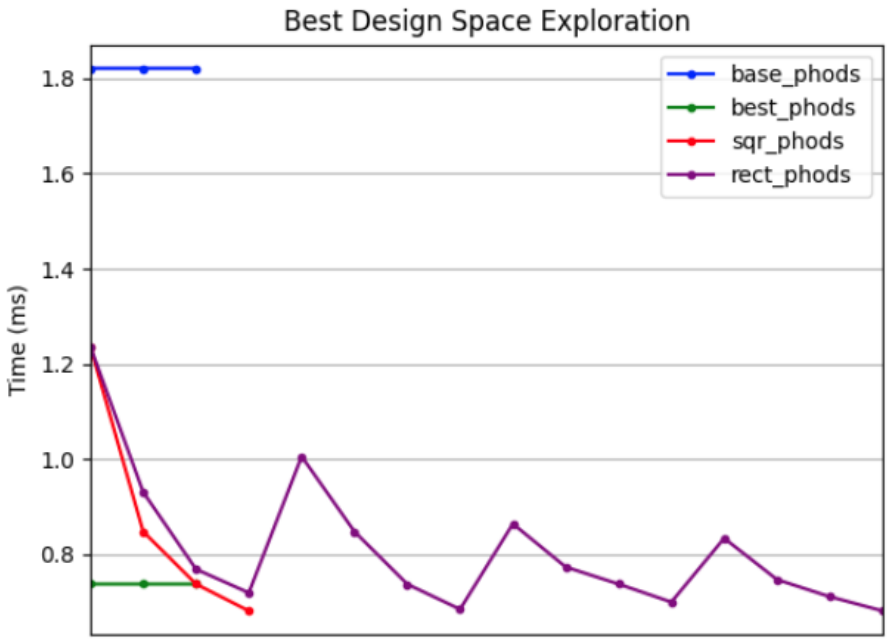
Bx=1	By=5	0.769116683 ms	0.769117032 ms	0.769120816 ms
Bx=1	By=10	0. 719620846 ms	0.719621661 ms	0.71962917 ms
Bx=2	By=1	1.00595411 ms	1.00595539 ms	1.00596668 ms
Bx=2	By=2	0.848137483 ms	0.848138356 ms	0.848145806 ms
Bx=2	By=5	0.737766677 ms	0.737767492 ms	0.737775001 ms
Bx=2	By=10	0.684941653 ms	0.684942876 ms	0.684954168 ms
Bx=5	By=1	0.864458329 ms	0.864458736 ms	0.86446252 ms
Bx=5	By=2	0.773754145 ms	0.773755019 ms	0.773762527 ms
Bx=5	By=5	0.738008355 ms	0.738009578 ms	0.738020812 ms
Bx=5	By=10	0.70006249 ms	0.700062898 ms	0.700066681 ms
Bx=10	By=1	0.834158331 ms	0.834159204 ms	0.834166654 ms
Bx=10	By=2	0.747362501 ms	0.747363723 ms	0.747375016 ms
Bx=10	By=5	0.711379165 ms	0.711379573 ms	0.711383356 ms

Bx=10	By=10	0.681566657 ms	0.681567879 ms	0.681579171 ms

Παρατηρούμε ότι η υλοποίηση με τον μικρότερο χρόνο εκτέλεσης είναι για **Bx=10 και By=10**.

Λόγω περιορισμένων πόρων του board είχαμε τιμές N=10 και M=10 οπότε η εξαντλητική αναζήτηση δεν ήταν πολύ χρονοβόρα. Στην περίπτωση όμως που είχαμε το τυπικό format που ορίζεται από το QCIF (176 x 144), η χρήση μιας ευριστικής μεθόδου μπορεί να μας οδηγούσε πιο γρήγορα σε καλό αποτέλεσμα. Παίρνοντας τυχαίες τιμές του block size θα βλέπαμε σε ποια περίπτωση έχουμε μικρότερο χρόνο εκτέλεσης και σε αυτή θα κάναμε εξαντλητική αναζήτηση.

Το line plot των ερωτημάτων 2-5 που προέκυψε από το `lineplot.py` φαίνεται παρακάτω:



Από το παραπάνω διαγράμματα μπορούμε να έχουμε μια συνολική εικόνα των επιδόσεων κάθε υλοποίησης προκειμένου να συγκρίνουμε και να αναλύσουμε τα αποτελέσματα τους. Αρχικά, παρατηρούμε ότι η βελτιστοποίηση του αλγορίθμου PHODS οδηγεί σε σημαντική ελάττωση του χρόνου εκτέλεσης. Έπειτα, βλέπουμε ότι η υλοποίηση του αλγορίθμου με τετραγωνικό μπλοκ μειώνει περαιτέρω τον χρόνο εκτέλεσης, όπως αντίστοιχα και η υλοποίηση με ορθογώνιο μπλοκ. Στην δική μας περίπτωση, επειδή το μέτωπο αναζήτησης μεγέθους μπλοκ ήταν μικρό και οι δυο υλοποιήσεις δίνουν την ίδια βέλτιστη επίδοση (sqr_phods→B=10, rect_phods→Bx=10, By=10). Σε μεγαλύτερο μέτωπο, όμως, θα αναμέναμε η ορθογώνια υλοποίηση να βρει

καλύτερη λύση λόγω των περισσότερων δοκιμών που γίνονται, αφού άλλωστε περιλαμβάνει και το μέτωπο της τετραγωνικής υλοποίησης. Τέλος, είναι αξιοσημείωτο ότι οι χρόνοι για B=5 και για B=10 για τις διαφορετικές υλοποιήσεις, ενώ θα περιμέναμε να είναι ίσοι σε κάθε περίπτωση, διαφέρουν λίγο μεταξύ τους. Αυτό μπορεί να οφείλεται σε εξωτερικούς παράγοντες που επικρατούσαν στο board κατά την εκτέλεση των προγραμμάτων όπως άλλες διεργασίες, διαφορετικοί χρόνοι πρόσβασης στην flash, διαφορές στα cache hits/misses, ασύγχρονες διακοπές, κλπ.

1.3 Ζητούμενο 2ο: Βελτιστοποίηση εφαρμογής σε διαφορετικές αρχιτεκτονικές x86

Το ζητούμενο σε αυτό το project είναι η βελτιστοποίηση της εφαρμογής `tables.c`, η οποία αφορά απλές προσπελάσεις και πράξεις σε πίνακες. Η εκτέλεση της εφαρμογής γίνεται σε δύο διαφορετικές **x86** αρχιτεκτονικές με τη χρήση του προσομοιωτή **Gem5**. Η πρώτη αρχιτεκτονική περιλαμβάνει μόνο τον επεξεργαστή και την κύρια μνήμη, ενώ η δεύτερη διαθέτει επιπλέον L1 Instruction, L1 Data και L2 cache.

1. Ακολουθώντας τις οδηγίες που παρέχονται στο GitHub repository, εγκαθιστούμε επιτυχώς τον προσομοιωτή και εκτελούμε την ακόλουθη εντολή:

```
$ build/X86/gem5.opt configs/learning_gem5/part1/simple.py
/gem5/tables_UF/tables.exe
```

Αυτή η εντολή προσομοιώνει την εφαρμογή `tables.exe` στην πρώτη αρχιτεκτονική, δηλαδή χωρίς καμία cache μνήμη. Το πρώτο ζητούμενο είναι να καταγράψουμε τους κύκλους ρολογιού που απαιτούνται για την ολοκλήρωση της εκτέλεσης. Αυτή η πληροφορία βρίσκεται στον φάκελο `m5out`, μέσα στο αρχείο `stats.txt`. Συγκεκριμένα, η μεταβλητή `system.cpu.numCycles` καταγράφει τον αριθμό κύκλων.

Για να αντλήσουμε αυτή την πληροφορία, χρησιμοποιούμε την εντολή:

```
$ cat m5out/stats.txt | grep "system.cpu.numCycles"
```

```
root@829fde635707:/gem5# build/X86/gem5.opt configs/learning_gem5/part1/simple.py /gem5/tables_UF/tables.exe
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 24.0.0.1
gem5 compiled Nov  5 2024 18:20:51
gem5 started Nov 11 2024 18:33:26
gem5 executing on 829fde635707, pid 132
command line: build/X86/gem5.opt configs/learning_gem5/part1/simple.py /gem5/tables_UF/tables.exe

Global frequency set at 1000000000000 ticks per second
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000
Beginning simulation!
src/sim/simulate.cc:199: info: Entering event queue @ 0.  Starting simulation...
src/sim/mem_state.cc:448: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)

Exiting @ tick 857212644000 because exiting with last active thread context
root@829fde635707:/gem5#
root@829fde635707:/gem5# cat m5out/stats.txt | grep "system.cpu.numCycles"
system.cpu.numCycles                        857212644                       # Number of cpu cycles simulated (Cycle)
root@829fde635707:/gem5#
```

Η προσομοίωση εκτέλεσης του προγράμματος `tables.exe` στην πρώτη αρχιτεκτονική ολοκληρώθηκε με επιτυχία και απαιτήθηκαν **857,212,644** (σύμφωνα με την ένδειξη της μεταβλητής `system.cpu.numCycles`).

Το simple.py αρχείο που χρησιμοποιήσαμε είναι το διορθωμένο όπως δόθηκε στο Forum του μαθήματος στο Helios το έτος 2023-2024.

2. Στο δεύτερο ερώτημα θα προσομοιώσουμε τη λειτουργία της εφαρμογής για την δεύτερη αρχιτεκτονική μέσω της εντολής:

```
$ build/X86/gem5.opt configs/learning_gem5/part1/two_level.py
/gem5/tables_UF/tables.exe --l1i_size=8kB --l1d_size=8kB --
l2_size=128kB
```

Αναζητώντας πάλι την μεταβλητή `system.cpu.numCycles` βρίσκουμε:

```
root@7a43f3774665:/gem5# cd m5out/
root@7a43f3774665:/gem5/m5out# cat stats.txt | grep system.cpu.numCycles
system.cpu.numCycles                        59787087                       # Number of cpu cycles simulated (Cycle)
root@7a43f3774665:/gem5/m5out#
```

Η προσομοίωση εκτέλεσης του προγράμματος `tables.exe` στην δεύτερη αρχιτεκτονική ολοκληρώθηκε με επιτυχία και απαιτήθηκαν **59,787,087** κύκλοι (σύμφωνα με την ένδειξη της μεταβλητής `system.cpu.numCycles`).

Η εισαγωγή της μνήμης cache στην αρχιτεκτονική του συστήματος, όπως αναμενόταν, μείωσε σημαντικά τους απαιτούμενους κύκλους για την εκτέλεση του προγράμματος. Αυτή η βελτίωση

αποδίδεται στα πλεονεκτήματα που προσφέρει η cache, μια εξαιρετικά ταχεία μνήμη, η οποία βρίσκεται κοντά στον πυρήνα του επεξεργαστή και είναι σχεδόν μία τάξη μεγέθους ταχύτερη από την κύρια μνήμη (RAM).

Η cache επιτρέπει στον επεξεργαστή γρηγορότερη πρόσβαση στα δεδομένα, καθώς τα δεδομένα που χρειάζονται συχνότερα προφορτώνονται στη μνήμη cache μέσω cache lines (ή cache blocks). Κάθε cache line μεταφέρει μαζί της και γειτονικά δεδομένα λόγω της αρχής της τοπικότητας, που χαρακτηρίζεται από τη χωρική και χρονική συγκέντρωση δεδομένων σε προγράμματα με επαναλαμβανόμενες προσπελάσεις, όπως το `tables.c`. Η εγγύτητα αυτών των δεδομένων εξασφαλίζει πως ο επεξεργαστής θα έχει άμεση πρόσβαση σε δεδομένα που είναι πιθανό να χρησιμοποιηθούν άμεσα, ελαχιστοποιώντας έτσι το κόστος των επανειλημμένων ανακτήσεων από την κύρια μνήμη.

Η προσπέλαση στην cache μειώνει σημαντικά τους κύκλους ρολογιού σε σύγκριση με την απευθείας πρόσβαση στη RAM, όπως συμβαίνει στην πρώτη αρχιτεκτονική. Η τοπικότητα που χαρακτηρίζει την εφαρμογή `tables.c` την καθιστά ιδανική για εκμετάλλευση της μνήμης cache, επιτρέποντας την επιτάχυνση των προσπελάσεων και αναθέσεων σε πίνακες και περιορίζοντας τις καθυστερήσεις της κύριας μνήμης.

3. Στο επόμενο βήμα της άσκησης, προσομοιώσαμε την εφαρμογή `tables.c` χρησιμοποιώντας διαφορετικούς **unrolling factors** (2, 4, 8, 16 και 32). Η τεχνική του loop unrolling επιτρέπει στον επεξεργαστή να μειώνει τις ελεγκτικές εντολές και να εκτελεί περισσότερες επαναλήψεις ενός βρόχου σε κάθε βήμα, βελτιώνοντας έτσι την απόδοση του προγράμματος με λιγότερους κύκλους ρολογιού.

Η προσομοίωση πραγματοποιήθηκε με την ακόλουθη εντολή:

```
$ build/X86/gem5.opt configs/learning_gem5/part1/two_level.py
/gem5/tables_UF/tables_ufXXX.exe --l1i_size=8kB --l1d_size=8kB -
l2_size=128kB
```

όπου το XXX αντιπροσωπεύει τον εκάστοτε παράγοντα unrolling

Για να αυτοματοποιήσουμε την διαδικασία των προσομοιώσεων, γράφουμε ένα script το οποίο θα εκτελεί για εμάς τις προσομοιώσεις για τις διάφορες τιμές των unrolling factors:

```
#!/bin/bash
```

```
# Define the unrolling factors
UNROLLING_FACTORS=(2 4 8 16 32)

# Define the path to the gem5 executable and the config file
GEM5_EXEC="build/X86/gem5.opt"
CONFIG_FILE="configs/learning_gem5/part1/two_level.py"
BASE_EXECUTABLE="/gem5/tables_UF/tables_uf"

# Define cache sizes
L1I_SIZE="8kB"
L1D_SIZE="8kB"
L2_SIZE="128kB"

# Output CSV file
OUTPUT_CSV="simulation_results.csv"

# Create CSV file and write header
echo "Unrolling Factor,NumCycles" > $OUTPUT_CSV

# Run simulations for each unrolling factor
for UF in "${UNROLLING_FACTORS[@]}"; do
    echo "Running simulation for Unrolling Factor = $UF..."
    # Construct the executable path
    EXECUTABLE="${BASE_EXECUTABLE}${UF}.exe"


 # Run the simulation
    $GEM5_EXEC $CONFIG_FILE $EXECUTABLE --l1i_size=$L1I_SIZE --
l1d_size=$L1D_SIZE --l2_size=$L2_SIZE

    # Extract the numCycles value from stats.txt
    NUM_CYCLES=$(grep "system.cpu.numCycles" m5out/stats.txt | awk '{print
$2}')

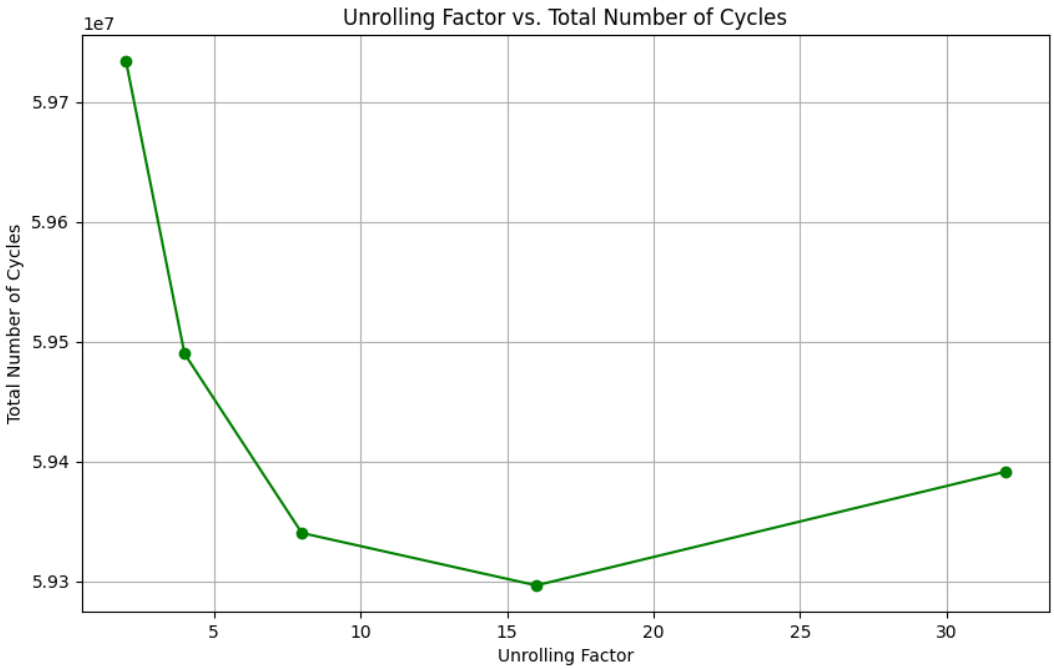
    # Append the result to the CSV file
    echo "$UF,$NUM_CYCLES" >> $OUTPUT_CSV
    echo "Unrolling Factor $UF: NumCycles = $NUM_CYCLES"
done

echo "All simulations completed. Results saved in '$OUTPUT_CSV'."
```

Μέσω αυτού του script, αποθηκεύουμε τα αποτελέσματα των προσομοιώσεων στο αρχείο `simulation_results.csv`. Τα περιεχόμενα του αρχείου φαίνονται παρακάτω:

```
root@adeda41156a7:/gem5#cat simulation_results.csv
Unrolling Factor,NumCycles
2,59734252
4,59489841
8,59340239
16,59296471
32,59391341
```

Το διάγραμμα που προκύπτει από τις μετρήσεις αυτές:



Το loop unrolling, όπως αναφέραμε και παραπάνω, είναι μια τεχνική μετασχηματισμού βρόχων που στοχεύει στην αύξηση της απόδοσης, μειώνοντας τους ελέγχους branch και επιτρέποντας, σε κάποιες περιπτώσεις, την παράλληλη εκτέλεση των εντολών μέσα στον βρόχο. Όσο μεγαλύτερος είναι ο unrolling factor, τόσο περισσότερα βήματα του βρόχου εκτελούνται σε μία επανάληψη, μειώνοντας τους κύκλους που απαιτούνται.

Ωστόσο, όσο αυξάνεται ο unrolling factor, ο κώδικας μεγαλώνει και η πολυπλοκότητα αυξάνεται, κάτι που μπορεί να επιφέρει μειωμένη απόδοση σε υψηλές τιμές unrolling. Στα αποτελέσματα των προσομοιώσεων με παράγοντες unrolling [2, 4, 8, 16, 32], παρατηρούμε αρχικά μια μείωση στους κύκλους ρολογιού, αλλά σε μεγαλύτερους παράγοντες αυτή η βελτίωση δεν κλιμακώνεται περαιτέρω. Αυτή η μείωση της απόδοσης εξηγείται από τα μειονεκτήματα του loop unrolling, όπως η αύξηση του μεγέθους και της πολυπλοκότητας του κώδικα στον βρόχο.

Στην συνέχεια μας ζητείται από την άσκση να εξερευνήσουμε την επίδραση διαφορετικών μεγεθών L1 Data cache (2kB, 4kB, 8kB, 16kB, 32kB και 64kB) για τη δεύτερη αρχιτεκτονική μέσω της εντολής:

```
$ build/X86/gem5.opt configs/learning_gem5/part1/two_level.py
/gem5/tables_UF/tables.exe --l1i_size=8kB --l1d_size=XXX -
l2_size=128kB
```

Όπως και στην προηγούμενη περίπτωση, αυτοματοποιούμε την διαδικασία εκτέλεσης των εντολών:

```
#!/bin/bash

# Define the L1 Data Cache Sizes
L1D_SIZES=("2kB" "4kB" "8kB" "16kB" "32kB" "64kB")

# Define the path to the gem5 executable and the config file
GEM5_EXEC="build/X86/gem5.opt"
CONFIG_FILE="configs/learning_gem5/part1/two_level.py"
EXECUTABLE="/gem5/tables_UF/tables.exe"

# Define other cache sizes
L1I_SIZE="8kB"
L2_SIZE="128kB"

# Output CSV file
OUTPUT_CSV="l1d_simulation_results.csv"

# Create CSV file and write header
echo "L1D Size,NumCycles" > $OUTPUT_CSV

# Run simulations for each L1D size
for L1D_SIZE in "${L1D_SIZES[@]}"; do
    echo "Running simulation for L1D Size = $L1D_SIZE..."

    # Run the simulation
    $GEM5_EXEC $CONFIG_FILE $EXECUTABLE --l1i_size=$L1I_SIZE --
l1d_size=$L1D_SIZE --l2_size=$L2_SIZE

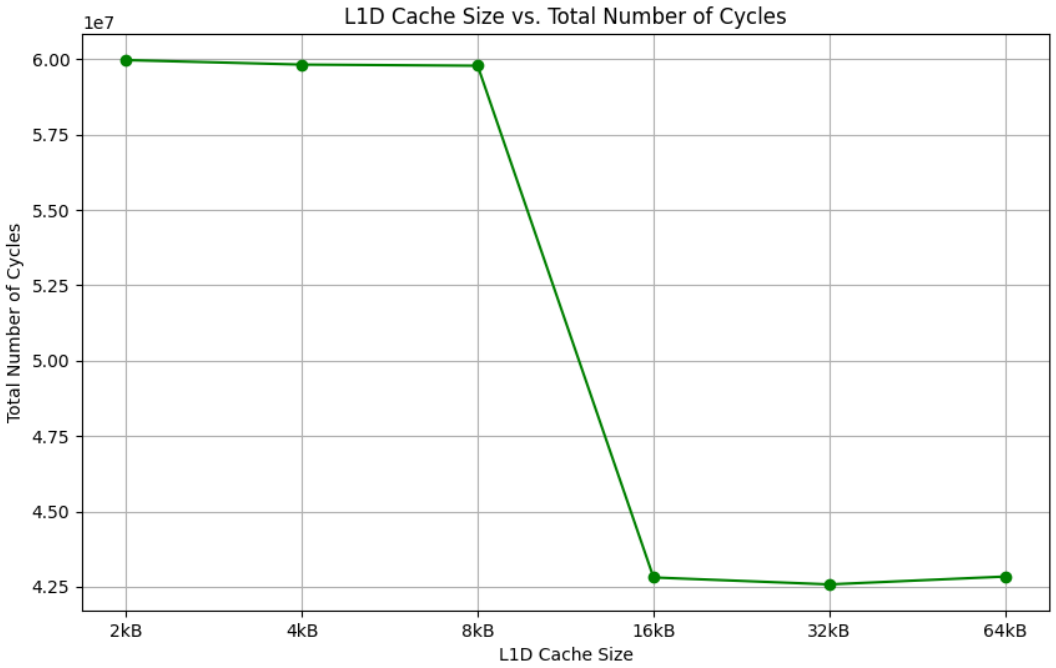
    # Extract the numCycles value from stats.txt
```

```
    NUM_CYCLES=$(grep "system.cpu.numCycles" m5out/stats.txt | awk
'{print $2}')

    # Append the result to the CSV file
    echo "$L1D_SIZE,$NUM_CYCLES" >> $OUTPUT_CSV
    echo "L1D Size $L1D_SIZE: NumCycles = $NUM_CYCLES"
done

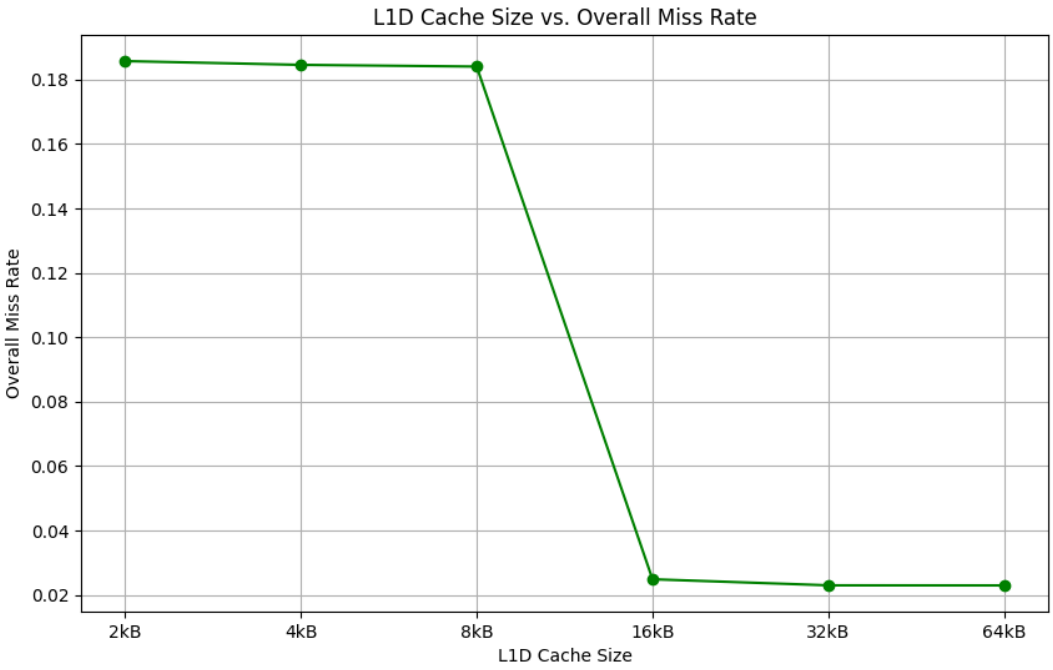
echo "All simulations completed. Results saved in '$OUTPUT_CSV'."
```

Τα αποτελέσματα αυτού αποθηκεύονται στο αρχείο l1d_simulation_results.csv. Τα διαγράμματα φαίνονται παρακάτω:



Όπως αναμενόταν, η αύξηση του μεγέθους της L1 data cache οδηγεί σε μείωση των απαιτούμενων κύκλων εκτέλεσης, καθώς η cache μπορεί να αποθηκεύσει μεγαλύτερο όγκο δεδομένων, μειώνοντας τις επισκέψεις στην αργότερη κύρια μνήμη (RAM). Για τις περιπτώσεις 2 kB, 4 kB, και 8 kB, οι κύκλοι είναι σχετικά κοντινοί. Αντίθετα, φαίνεται στις περιπτώσεις 16 kB, 32 kB και 64 kB, παρατηρείται σημαντική μείωση των κύκλων. Ωστόσο, δεν υπάρχει περαιτέρω βελτίωση όταν αυξάνεται η cache από 32 kB σε 64 kB· αντιθέτως, η απόδοση φαίνεται να μειώνεται ελαφρώς.
Ένα άλλο μέγεθος το οποίο έχει ενδιαφέρον να μετρήσουμε είναι το

`system.cpu.dcache.overallMissRate::cpu.data` για όλες τις περιπτώσεις της L1 data cache.



Παρατηρήσαμε ότι για μεγέθη cache 2 kB, 4 kB και 8 kB, το ποσοστό των cache misses ανέρχεται στο 18,5%. Στις μεγαλύτερες cache (16 kB, 32 kB και 64 kB), το ποσοστό αυτό μειώνεται σημαντικά στο 2,3%. Αυτό υποδεικνύει ότι στις μεγαλύτερες cache, ο πυρήνας έχει πρόσβαση σε μεγαλύτερα σύνολα δεδομένων εντός της cache, μειώνοντας έτσι τις ανακτήσεις από τη RAM.

Παρατηρώντας τα δεδομένα, διαπιστώνουμε ότι η αύξηση του μεγέθους της cache δεν οδηγεί απαραίτητα σε γραμμική βελτίωση της απόδοσης. Η σχέση μεταξύ μεγέθους cache και επίδοσης δεν είναι πάντα αντιστρόφως ανάλογη. Για παράδειγμα, ενώ η αύξηση από 2 kB σε 32 kB μειώνει αισθητά τους κύκλους, η μετάβαση στα 64kB δεν προσφέρει αντίστοιχη βελτίωση και εμφανίζει μικρή επιδείνωση στους κύκλους.

Αυτό μπορεί να εξηγηθεί από το trade-off μεταξύ μεγέθους cache και ταχύτητας πρόσβασης. Μια μεγαλύτερη cache ενδέχεται να προκαλέσει αυξημένο χρόνο πρόσβασης λόγω πολυπλοκότητας, επηρεάζοντας την απόδοση. Αντίθετα, μια μικρότερη cache έχει γρηγορότερη πρόσβαση αλλά περιορισμένη χωρητικότητα, κάτι που οδηγεί σε περισσότερα cache misses. Συνεπώς, το ιδανικό μέγεθος cache είναι μια ισορροπία ανάμεσα στη χωρητικότητα και τη γρηγορότερη απόκριση, με στόχο τη βέλτιστη απόδοση του συστήματος.

4. Στόχος του τελευταίου μέρους της άσκησης είναι η εύρεση της βέλτιστης αρχιτεκτονικής, η οποία συνδυάζει την υψηλή απόδοση της εφαρμογής με την καλύτερη δυνατή λύση ως προς τη συνολική μνήμη. Το πρόβλημα αυτό ανήκει στην κατηγορία των **multi-objective βελτιστοποιήσεων** και, συγκεκριμένα, αποτελεί ένα **min-min optimization πρόβλημα**, καθώς επιθυμούμε να ελαχιστοποιήσουμε τόσο τον αριθμό των κύκλων εκτέλεσης (latency) όσο και το συνολικό μέγεθος της μνήμης.

Για την επίλυση του προβλήματος αυτού, κατασκευάζουμε το **Pareto Frontier**, το οποίο περιέχει τις βέλτιστες λύσεις με το καλύτερο trade-off μεταξύ των δύο παραμέτρων, δηλαδή των κυκλικών καθυστερήσεων και της συνολικής μνήμης. Το Pareto Frontier μας δείχνει όλα τα σημεία που δεν μπορούν να «κυριαρχηθούν» από άλλα ως προς και τις δύο παραμέτρους. Η τελική επιλογή ανάμεσα στις Pareto optimal λύσεις ανήκει στον μηχανικό, ανάλογα με τις προτεραιότητες της αρχιτεκτονικής.

Μια πρώτη προσέγγιση για την εύρεση των Pareto efficient λύσεων είναι η εξαντλητική αναζήτηση, όπου εξετάζουμε όλους τους πιθανούς συνδυασμούς των παραμέτρων. Η μέθοδος αυτή είναι πλήρης και εξασφαλίζει ότι δεν θα παραλειφθεί κανένα πιθανό βέλτιστο σημείο. Ωστόσο, όταν το πρόβλημα περιέχει μεγάλο αριθμό παραμέτρων, η εξαντλητική αναζήτηση μπορεί να απαιτεί πολύ χρόνο για να ολοκληρωθεί. Στη συγκεκριμένη περίπτωση, η εξαντλητική αναζήτηση για όλους τους συνδυασμούς ολοκληρώθηκε σε 2 ώρες (η πληροφορία αυτή αντλήθηκε από τη μεταβλητή `hostSeconds` στο αρχείο `stats.txt`).

Η αναζήτηση πραγματοποιήθηκε μέσω ενός bash script που περιείχε τέσσερις nested for loops, τρέχοντας όλους τους πιθανούς συνδυασμούς για τις παρακάτω παραμέτρους:

- **L1D cache size** ∈ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
- **L1I cache size** ∈ [2kB, 4kB, 8kB, 16kB, 32kB, 64kB]
- **L2 cache size** ∈ [128kB, 256kB, 512kB, 1024kB]
- **Unrolling factor** ∈ [2, 4, 8, 16, 32]

Οι εντολές προσομοίωσης είχαν τη μορφή:

```
$ build/X86/gem5.opt configs/learning_gem5/part1/two_level.py
/gem5/tables_UF/tables_ufXXX.exe --l1i_size=XXX --l1d_size=XXX -
l2_size=XXX
#!/usr/bin/env python3
```

Το script φαίνεται παρακάτω:

```
import pandas as pd
import subprocess
import time
import os

# Define parameter ranges
l1d_sizes = ["2kB", "4kB", "8kB", "16kB", "32kB", "64kB"]
l1i_sizes = ["2kB", "4kB", "8kB", "16kB", "32kB", "64kB"]
l2_sizes = ["128kB", "256kB", "512kB", "1024kB"]
unrolling_factors = [2, 4, 8, 16, 32]

# DataFrame to store results in the required format
df = pd.DataFrame(columns=[
    'L1I_cache_size', 'L1D_cache_size', 'L2_cache_size',
    'unrolling_factor', 'exec_time_cc', 'total_mem_kB'])

# Track time taken for exhaustive search
start_time = time.time()

# Iterate over all configurations
for l1d in l1d_sizes:
    for l1i in l1i_sizes:
        for l2 in l2_sizes:
            for uf in unrolling_factors:
                # Construct command
                command = [
                    "build/X86/gem5.opt",
                    "configs/learning_gem5/part1/two_level.py",
                    f"/gem5/tables_UF/tables_uf{uf}.exe",
                    f"--l1i_size={l1i}",
                    f"--l1d_size={l1d}",
                    f"--l2_size={l2}"
                ]

                # Print the configuration and command for clarity
                print(f"\nRunning configuration: L1D={l1d},
L1I={l1i}, L2={l2}, Unrolling Factor={uf}")
                print(f"Command: {' '.join(command)}")
```

```
                # Run gem5 command and capture output
                try:
                    result = subprocess.run(command,
capture_output=True, text=True, check=True)

                    # Print the output from the gem5 command
                    print("Command Output:")
                    print(result.stdout)  # Shows the stdout output
from the gem5 command

                    # Path to the stats file
                    stats_file_path = "m5out/stats.txt"  # Adjust if
necessary

                    # Check if stats file exists
                    if not os.path.exists(stats_file_path):
                        print(f"Stats file not found for config
L1D={l1d}, L1I={l1i}, L2={l2}, UF={uf}")
                        continue

                    # Read the cycle count from stats.txt
                    exec_time_cc = None
                    with open(stats_file_path, 'r') as stats_file:
                        for line in stats_file:
                            if "system.cpu.numCycles" in line:  #
Updated keyword
                                exec_time_cc = int(line.split()[1])
# Parse the cycle count
                                print(f"Execution time
(system.cpu.numCycles): {exec_time_cc}")
                                break

                    # Check if execution time was found; skip if not
found
                    if exec_time_cc is None:
                        print(f"Execution time (system.cpu.numCycles)
not found in stats for config L1D={l1d}, L1I={l1i}, L2={l2},
UF={uf}")
```

```
                        continue

                    # Calculate total memory in KB
                    total_mem_kb = int(l1d[:-2]) + int(l1i[:-2]) +
int(l2[:-2])
                    print(f"Total memory for configuration:
{total_mem_kb} KB")

                    # Append the result in the required format
                    temp = {
                        'L1I_cache_size': l1i,
                        'L1D_cache_size': l1d,
                        'L2_cache_size': l2,
                        'unrolling_factor': str(uf),
                        'exec_time_cc': str(exec_time_cc),
                        'total_mem_kB': str(total_mem_kb)
                    }
                    df.loc[len(df)] = temp

                except subprocess.CalledProcessError as e:
                    print(f"Error running command for config
L1D={l1d}, L1I={l1i}, L2={l2}, UF={uf}")
                    print(e.output)

# Calculate and print execution time
execution_time = time.time() - start_time
print(f"\nTotal time for exhaustive search: {execution_time:.2f}
seconds")

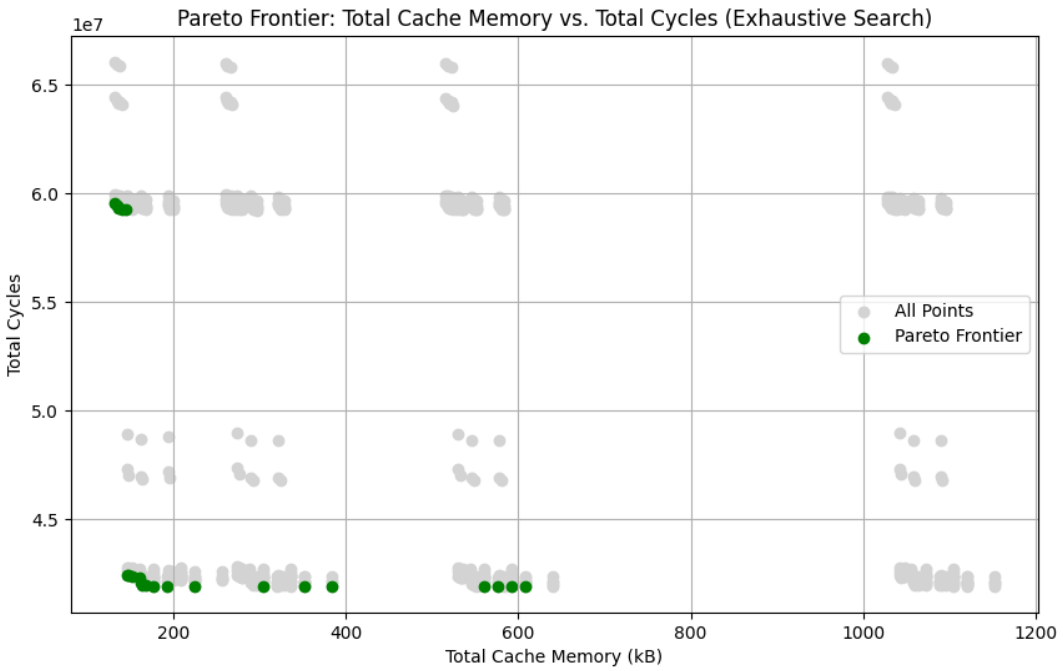
# Save results to CSV in the required format
df.to_csv("exhaustive_search_results_PO.csv", index=False)
print("\nResults saved to exhaustive_search_results_PO.csv")
```

Για την ανάλυση των αποτελεσμάτων, δημιουργήσαμε ένα **Pandas DataFrame** με δύο στήλες:

- `Total Cache Memory (kB)`, που περιλαμβάνει το άθροισμα της μνήμης σε KB για την εκάστοτε αρχιτεκτονική.
- `Total Cycles`, που περιέχει τον αριθμό των κύκλων εκτέλεσης.

Στη συνέχεια, χρησιμοποιώντας τη βιβλιοθήκη `paretoset`, εντοπίσαμε τα Pareto optimal σημεία και καταγράψαμε το Pareto Frontier. Στο αρχείο CSV με τις λύσεις του Pareto Frontier περιλαμβάνονται οι παραμετροποιήσεις της αρχιτεκτονικής που αντιστοιχούν στα βέλτιστα σημεία.

```
Pareto Optimal Solutions:
    L1I_cache_size L1D_cache_size L2_cache_size  unrolling_factor  exec_time_cc  total_mem_kB
2              2kB            2kB         128kB                 8      59581198           132
122            2kB            4kB         128kB                 8      59433698           134
143            4kB            4kB         128kB                16      59343511           136
263            4kB            8kB         128kB                16      59306017           140
283            8kB            8kB         128kB                16      59296471           144
362            2kB           16kB         128kB                 8      42416612           146
383            4kB           16kB         128kB                16      42409832           148
403            8kB           16kB         128kB                16      42403308           152
423           16kB           16kB         128kB                16      42342672           160
482            2kB           32kB         128kB                 8      42076537           162
503            4kB           32kB         128kB                16      41969420           164
523            8kB           32kB         128kB                16      41961440           168
543           16kB           32kB         128kB                16      41945801           176
548           16kB           32kB         256kB                16      41933039           304
553           16kB           32kB         512kB                16      41923846           560
563           32kB           32kB         128kB                16      41936488           192
573           32kB           32kB         512kB                16      41923699           576
583           64kB           32kB         128kB                16      41935485           224
588           64kB           32kB         256kB                16      41930295           352
673           16kB           64kB         512kB                16      41922998           592
693           32kB           64kB         512kB                16      41920339           608
708           64kB           64kB         256kB                16      41925024           384
root@adeda41156a7:/gem5#
```



Παρατηρούμε ότι τα Pareto optimal σημεία ακολουθούν την αναμενόμενη καμπύλη για ένα min-min optimization πρόβλημα. Ωστόσο, η εξαντλητική αναζήτηση, αν και εγγυάται την πλήρη εξερεύνηση του χώρου των λύσεων, εξετάζει και πολλές μη βέλτιστες λύσεις, καταναλώνοντας χρόνο σε συνδυασμούς που είναι προφανές ότι θα «κυριαρχηθούν» από άλλους. Το Pareto Frontier αποκαλύπτει ότι η αύξηση του μεγέθους της μνήμης cache ή άλλων παραμέτρων δεν βελτιώνει πάντα γραμμικά την απόδοση, καθώς υπάρχει ένα σημείο πέρα από το οποίο η προσθήκη μνήμης ή η αύξηση της πολυπλοκότητας δεν προσφέρει ουσιαστική βελτίωση και, σε ορισμένες περιπτώσεις, μπορεί να οδηγήσει σε επιδείνωση των επιδόσεων.

5. Μια δεύτερη προσέγγιση για την εύρεση των Pareto efficient λύσεων είναι η χρήση γενετικού αλγορίθμου. Αυτή η μέθοδος δεν εξετάζει όλους τους δυνατούς συνδυασμούς, αλλά μέσω της εξέλιξης και επιλογής απορρίπτει πολλά design points χωρίς να χρειάζεται να τα αξιολογήσει πλήρως. Παρόλο που η μέθοδος μπορεί να μην είναι απόλυτα ακριβής, προσφέρει καλής ποιότητας λύσεις σε πολύ λιγότερο χρόνο, ειδικά σε προβλήματα με σημαντικό αριθμό παραμέτρων. Στην περίπτωσή μας, η εκτέλεση με γενετικό αλγόριθμο ολοκληρώθηκε σε 20 λεπτά, έναντι των 2 ωρών της εξαντλητικής αναζήτησης.

Για την εκτέλεση του γενετικού αλγορίθμου χρησιμοποιήσαμε το διαθέσιμο Python script:

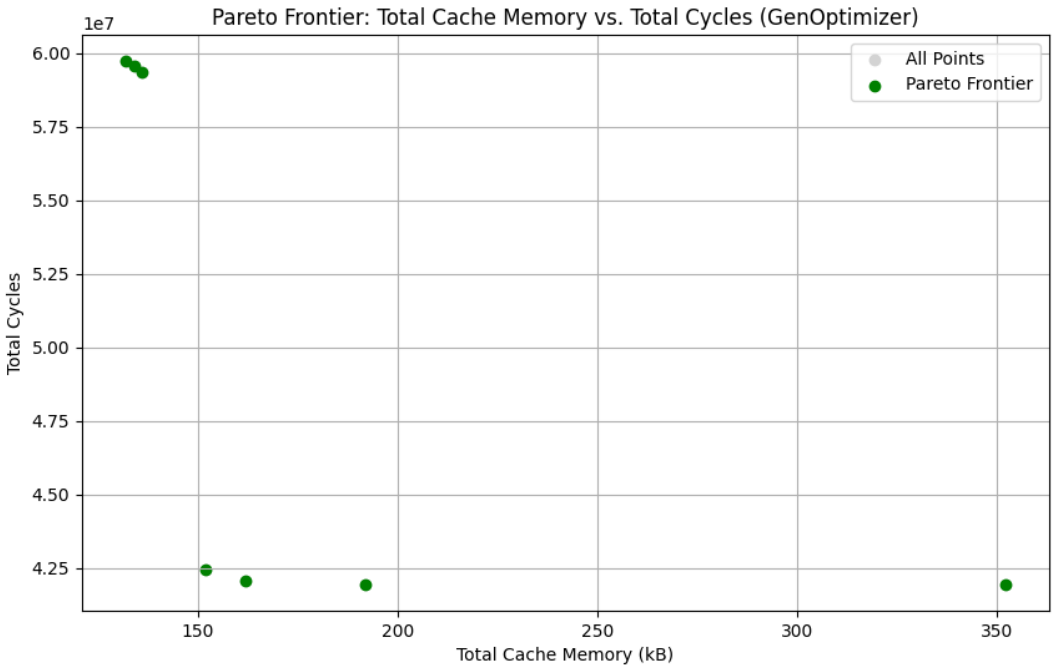
```
$ python3 genOptimizer.py
```

Μετά την εκτέλεση, λάβαμε το νέο Pareto Frontier, καθώς και τον αριθμό των αξιολογήσεων (evaluations) που πραγματοποίησε ο αλγόριθμος.

```
Pareto Optimal Configurations

1) L1I (KB) = 4 L1D (KB) = 4 L2 (KB) = 128 UF = 16 config. results in Execution Time (CC) = 59343511 and Total Memory (KB) = 136
2) L1I (KB) = 2 L1D (KB) = 4 L2 (KB) = 128 UF = 4 config. results in Execution Time (CC) = 59583529 and Total Memory (KB) = 134
3) L1I (KB) = 32 L1D (KB) = 64 L2 (KB) = 256 UF = 16 config. results in Execution Time (CC) = 41934202 and Total Memory (KB) = 352
4) L1I (KB) = 2 L1D (KB) = 32 L2 (KB) = 128 UF = 8 config. results in Execution Time (CC) = 42076537 and Total Memory (KB) = 162
5) L1I (KB) = 2 L1D (KB) = 2 L2 (KB) = 128 UF = 4 config. results in Execution Time (CC) = 59731958 and Total Memory (KB) = 132
6) L1I (KB) = 32 L1D (KB) = 32 L2 (KB) = 128 UF = 16 config. results in Execution Time (CC) = 41936488 and Total Memory (KB) = 192
7) L1I (KB) = 8 L1D (KB) = 16 L2 (KB) = 128 UF = 8 config. results in Execution Time (CC) = 42431214 and Total Memory (KB) = 152
root@7a43f3774665:/gem5#
```

Ο γενετικός αλγόριθμος παρήγαγε 7 Pareto solutions, τις οποίες αναλύσαμε χρησιμοποιώντας τη βιβλιοθήκη paretoset για να καταλήξουμε στο τελικό Pareto Frontier. Σημειώνεται ότι τα σημεία του Pareto ακολουθούν και πάλι την αναμενόμενη καμπύλη για ένα min-min optimization πρόβλημα, επιβεβαιώνοντας την ποιότητα της λύσης.



Ωστόσο, παρατηρούμε ότι ο γενετικός αλγόριθμος εντόπισε ένα υποσύνολο των βέλτιστων Pareto points της εξαντλητικής αναζήτησης (7 από τα 22). Επιπλέον, ο αλγόριθμος κατέληξε σε ένα σημείο (latencyCC = 41,934,202, totalMemoryKB = 352) που, αν και μέρος του Pareto Frontier, «κυριαρχείται» από τη λύση (latencyCC = 41,933,039, totalMemoryKB = 304), η οποία είχε απορριφθεί κατά την εξαντλητική αναζήτηση λόγω του ότι απαιτούσε περισσότερη μνήμη για ελαφρώς λιγότερη καθυστέρηση.

Η χρήση του γενετικού αλγόριθμου απέδειξε πως μπορεί να εντοπίσει ικανοποιητικές λύσεις σε πολύ λιγότερο χρόνο σε σύγκριση με την εξαντλητική αναζήτηση, αν και μερικά suboptimal σημεία μπορεί να περιλαμβάνονται στο τελικό Pareto Frontier. Συνολικά, η επιλογή της μεθόδου εξαρτάται από τον διαθέσιμο χρόνο και τους πόρους. Εάν είναι διαθέσιμοι, η εξαντλητική αναζήτηση εξασφαλίζει ότι καμία βέλτιστη λύση δεν θα παραβλεφθεί. Αν όμως η ταχύτητα είναι κρίσιμη και το πλήθος των παραμέτρων μεγάλο, ο γενετικός αλγόριθμος προσφέρει ένα αποδοτικό εργαλείο για την αναζήτηση βέλτιστων λύσεων.

Το script για την υλοποίηση των διαγραμμάτων φαίνεται παρακάτω:

```
import pandas as pd
import matplotlib.pyplot as plt
from paretoset import paretoset

# Function to plot line plot for Unrolling Factor vs. NumCycles
def plot_unrolling_factor_cycles():
    df = pd.read_csv('simulation_results.csv')
    plt.figure(figsize=(10, 6))
    plt.plot(df['Unrolling Factor'], df['NumCycles'], marker='o',
color='green', linestyle='-')
    plt.xlabel('Unrolling Factor')
    plt.ylabel('Total Number of Cycles')
    plt.title('Unrolling Factor vs. Total Number of Cycles')
    plt.grid(True)
    plt.savefig("unrolling_factor_vs_cycles.png", format="png")
    plt.show()

# Function to plot line plot for L1D Cache Size vs. NumCycles
def plot_l1d_cache_size_cycles():
    df = pd.read_csv('l1d_simulation_results.csv')
    plt.figure(figsize=(10, 6))
    plt.plot(df['L1D Size'], df['NumCycles'], marker='o',
color='green', linestyle='-')
    plt.xlabel('L1D Cache Size')
    plt.ylabel('Total Number of Cycles')
    plt.title('L1D Cache Size vs. Total Number of Cycles')
    plt.grid(True)
    plt.savefig("l1d_cache_size_vs_cycles.png", format="png")
    plt.show()

# Function to plot line plot for L1D Cache Size vs. Miss Rate
def plot_l1d_cache_size_miss_rate():
    df = pd.read_csv('l1d_simulation_results.csv')
    plt.figure(figsize=(10, 6))
    plt.plot(df['L1D Size'], df['OverallMissRate'], marker='o',
color='green', linestyle='-')
    plt.xlabel('L1D Cache Size')
```

```
    plt.ylabel('Overall Miss Rate')
    plt.title('L1D Cache Size vs. Overall Miss Rate')
    plt.grid(True)
    plt.savefig("l1d_cache_size_vs_miss_rate.png", format="png")
    plt.show()

# Function to plot Pareto frontier for Exhaustive Search Results
def plot_pareto_exhaustive_search():
    df = pd.read_csv('exhaustive_search_results_PO.csv')
    df['total_cache_memory_kB'] = (
        df['L1I_cache_size'].str.replace('kB', '').astype(int) +
        df['L1D_cache_size'].str.replace('kB', '').astype(int) +
        df['L2_cache_size'].str.replace('kB', '').astype(int)
    )

    # Pareto set
    pareto_mask = paretoset(df[['exec_time_cc',
'total_cache_memory_kB']], sense=['min', 'min'])
    pareto_front = df[pareto_mask]

    plt.figure(figsize=(10, 6))
    plt.scatter(df['total_cache_memory_kB'], df['exec_time_cc'],
color='lightgray', label='All Points')
    plt.scatter(pareto_front['total_cache_memory_kB'],
pareto_front['exec_time_cc'], color='green', label='Pareto Frontier')
    plt.xlabel('Total Cache Memory (kB)')
    plt.ylabel('Total Cycles')
    plt.title('Pareto Frontier: Total Cache Memory vs. Total Cycles
(Exhaustive Search)')
    plt.legend()
    plt.grid(True)
    plt.savefig("pareto_frontier_exhaustive_search.png",
format="png")
    plt.show()

# Function to plot Pareto frontier for GenOptimizer Results
def plot_pareto_genoptimizer():
    df = pd.read_csv('genOptimizer_PO.csv')
    df['total_cache_memory_kB'] = (
```

```
        df['L1I_cache_size'].str.replace('kB', '').astype(int) +
        df['L1D_cache_size'].str.replace('kB', '').astype(int) +
        df['L2_cache_size'].str.replace('kB', '').astype(int)
    )

    # Pareto set
    pareto_mask = paretoset(df[['exec_time_cc',
'total_cache_memory_kB']], sense=['min', 'min'])
    pareto_front = df[pareto_mask]

    plt.figure(figsize=(10, 6))
    plt.scatter(df['total_cache_memory_kB'], df['exec_time_cc'],
color='lightgray', label='All Points')
    plt.scatter(pareto_front['total_cache_memory_kB'],
pareto_front['exec_time_cc'], color='green', label='Pareto Frontier')
    plt.xlabel('Total Cache Memory (kB)')
    plt.ylabel('Total Cycles')
    plt.title('Pareto Frontier: Total Cache Memory vs. Total Cycles
(GenOptimizer)')
    plt.legend()
    plt.grid(True)
    plt.savefig("pareto_frontier_genoptimizer.png", format="png")
    plt.show()

# Main function to run all plots
def main():
    plot_unrolling_factor_cycles()
    plot_l1d_cache_size_cycles()
    plot_l1d_cache_size_miss_rate()
    plot_pareto_exhaustive_search()
    plot_pareto_genoptimizer()

if __name__ == '__main__':
    main()
```